

AI Agent 时代的人机协同软件工程：一次深度探索

摘要

本文记录了一次针对 AI Agent 软件工程实践的系统性探索。通过一个具体案例研究——在 oc-collab 项目中发现并分析一个功能缺失问题——我们展示了**传统软件工程在 AI Agent 场景下的适用性**。

这次探索最重要的收获不在于技术方案，而在于揭示了**传统软件工程的价值与人类执行困境之间的关系**：传统软件工程的设计假设本身是合理的——多人协作确实需要流程控制、文档规范、质量门禁。问题在于人类员工普遍抵触文档工作，导致流程失去载体。但这并不意味着流程本身有问题，而是人类执行层面的问题。

AI Agent 不会拒绝工作，传统软件工程的所有经验和设想都可以在 AI Agent 开发中找到用武之地。核心问题从"如何让人类愿意执行流程"转变为"如何通过强制约束机制确保 AI Agent 完整执行流程"。

摘要（浓缩版）

本文记录了一次针对 AI Agent 软件工程实践的系统性探索，通过 oc-collab 项目中"session_start"功能缺失的真实案例，展示**传统软件工程在 AI Agent 场景下的适用性**。

核心发现：传统软件工程的设计假设本身是合理的——多人协作确实需要流程控制、文档规范、质量门禁。问题在于人类员工普遍抵触文档工作，导致流程失去载体。但这并不意味着流程本身有问题，而是人类执行层面的问题。

关键转折：最初的 5-Why 分析错误地得出"传统软件工程假设对 AI 不成立"的结论，但被人类专家质疑——"人就一定会主动发现问题吗？"这个质疑引导我们找到真正的根因：人类的问题在于主观抵触（不愿执行），**AI 的问题在于技术限制（context rotating、幻觉）**。传统软件流程对 **AI 完全适用**，只需解决技术层面的约束执行问题。

核心洞察：不是"修正"传统软件工程的假设，而是"发现"传统软件工程在 AI 场景下可以完美适用。AI Agent 不会拒绝执行流程，问题是如何通过强制约束机制确保流程被完整执行。

解决方案：四项核心原则（强制约束、系统自动验证、追溯关联、持久化记忆）和动态 checklist 机制，确保 AI Agent 工作流程的完整性和可追溯性。

背景简介

oc-collab 是什么

oc-collab 是一个双 **Agent** 协作开发框架，用于协调两个 AI Agent（产品经理 Agent 1 + 开发 Agent 2）共同完成软件项目。它借鉴传统软件工程的阶段划分（需求→设计→开发→测试→部署），通过里程碑签署、质量门禁、状态管理等机制，确保开发过程规范化。

v2.2.0 核心功能

| 功能 | 说明 |

|-----|-----|

| 双 Agent 角色管理 | Agent 1（产品）+ Agent 2（开发），职责分离 |

| 里程碑签署 | 每个阶段完成后需双方签字确认 |

| 状态管理 | 记录项目当前 phase、签署状态、待办事项 |

| 记忆机制 | 记录决策、问题模式、经验教训 |

极其简单的使用流程

详细背景资料请见本文档末尾的"附录：oc-collab 项目背景"。

第一章：问题的起点

1.1 一个沉默的失败

2026 年 2 月，在与一个金融文档生成系统项目中的 AI Agent 进行常规对话时，我们发现了一个令人担忧的问题：该 Agent 对其应该使用的协作框架（oc-collab）毫无认知。当被问及项目工作流程时，该 Agent 的回答仿佛它在独立工作，浑然不知本应存在的角色定义、阶段转换或质量门禁。

这不是一个小的疏漏，而是项目流程控制中的根本性崩溃。

1.2 缺失的功能

调查后，我们发现 oc-collab 的 v2.2.0 需求文档中有一个名为 "session_start" 的功能，旨在在每次会话开始时为 Agent 提供上下文信息。需求文档中写道：

"每次会话开始 | Agent 启动 | '上次会话遗留问题: 3个'"

然而，当我们检查交付的产品时，这个功能并不存在。没有设计文档，没有测试用例，没有实现代码——什么都没有。需求存在，但功能不存在。

1.3 为什么这很重要

这不仅仅是一个待修复的 bug，而是一个更深层问题的症状：

- 如果需求可以写下但永远不被实现，流程就已经坏了
- 如果多个评审阶段（需求评审、设计评审、测试评审、签署）都能在不发现这个缺失的情况下完成，评审就毫无意义
- 如果一个核心功能的缺失能直到 Agent 的随意评论才被发现，质量控制机制就是虚幻的

问题不再是"我们如何修复这个 bug？"，而是"我们的整个流程怎么会没能防止这种情况？"

第二章：调查过程

2.1 追溯责任链条

我们对所有项目阶段进行了系统性调查：

阶段	文档/代码	存在?	问题
需求	requirements_v2.2.0.md	✓	有描述但不完整，缺乏具体性
设计	docs/02-design/*.md	✗	没有任何设计文档涉及此功能
测试	docs/03-test/*.md	✗	不存在测试用例
代码	src/*.py	✗	没有实现
签署	M1-M5 reviews	✓	全部签署；无人发现缺失

2.2 来自一线的证据

我们使用命令行工具验证了发现：

该功能只存在于一个地方：需求文档。它从一开始就注定不会实现。

第三章：5-Why 分析的修正——人机协作的核心价值

3.1 第一次 5-Why 分析（错误版本）

我最初进行了 5-Why 分析，得到的“根因”是：

"oc-collab 用'人的协作方式'设计 AI Agent 流程，但 AI Agent 不会主动检查、不会主动验证、不会主动发现缺失。"

这个分析的隐含假设是：“传统软件工程假设对 AI 不成立，需要完全不同的设计。”

3.2 人类专家的质疑

你指出了我分析中的根本性错误：

“实际上面向人类开发团队的传统软件工程其起源原本就是为了对多人团队开展软件开发作出有效控制。在实操中，因为人类员工普遍极端抵触大量的文档工作，导致文档更新不及时，文档残缺不全等等问题，是文档变得无用，而流程失去载体。在人类软件工程中，使用简单流程的原因并非人类不需要，而是人类会拒绝和消极抵抗。但 AI 不会。因此，人类关于软件工程，干预流程管控，所有的经验和设想，实际上都可以在 AI Agent 的开发工作中找到用武之地：毕竟，AI Agent 不遵循指令主要是因为 context rotating 或者幻觉，而不是主观意愿上的不愿意。”

这是一个极其深刻的修正。

3.3 真正的洞察

经过讨论，我们共同修正了分析。**传统软件工程的设计假设本身是合理的：**

| 方面 | 传统软件工程的假设 | 实际情况 |

|-----|-----|-----|

| 设计层面 | 多人协作需要流程控制 |  确实需要 |

| 文档层面 | 文档是协作的载体 |  确实是载体 |

| 执行层面 | 人类会执行流程 |  人类抵触执行 |

| 质量层面 | 需要质量门禁 |  确实需要 |

人类的问题：主观意愿上的不愿意——抵触文档工作、消极抵抗流程。

AI Agent 的问题：技术限制——context rotating（上下文丢失）、幻觉，而非主观不愿意。

核心结论：

不是“修正”传统软件工程的假设，而是“发现”传统软件工程在 AI 场景下可以完美适用。

人类关于软件工程、流程管控的所有经验和设想，都可以在 *AI Agent* 开发工作中找到用武之地。

核心问题从“如何让人类愿意执行流程”转变为“如何通过强制约束机制确保 *AI Agent* 完整执行流程”。

3.4 人类的核心价值

虽然传统软件工程的流程设计可以直接应用于 *AI Agent*，但人类的价值依然不可替代：

- **质疑能力**：人类能够质疑隐含的假设，指出逻辑漏洞
- **判断力**：人类能够判断什么是对的、什么是错的，即使没有技术细节
- **引导方向**：人类能够引导 AI 走向正确的研究方向，而非让 AI 在错误假设下越走越远

在这次探索中，你（人类专家）没有关注 `grep` 命令的结果、代码结构、技术实现等细节，但你凭借对软件工程本质的理解，指出“传统软件工程假设本身是对的，人类不执行是因为抵触，而非假设错误”。这直接引导我们找到了正确的方向。

第四章：AI Agent 软件工程的根本性假设修正

4.1 传统假设的错误

我最初的分析错误地假设传统软件工程的假设对 AI Agent 不成立，需要完全不同的设计。

4.2 修正后的理解

经过人类专家的质疑和讨论，我们认识到：

传统软件工程的设计假设本身是合理的：

| 设计假设 | 是否合理 | 人类执行问题 | AI Agent 问题 |

|-----|-----|-----|-----|

| 多人协作需要流程控制 | ☒ | 抵触执行 | context rotating |

| 文档是协作的载体 | ☒ | 更新不及时 | 幻觉 |

| 需要质量门禁 | ☒ | 绕过门禁 | 需要强制约束 |

关键区别：

- 人类的问题：**主观不愿意**——抵触文档工作、消极抵抗流程
- AI Agent 的问题：**技术限制**——context rotating（上下文丢失）、幻觉，而非主观不愿意

核心洞察：

传统软件工程起源于对多人团队的控制，设计假设本身是正确的。人类员工普遍抵触文档工作，导致流程失去载体——这是人类执行层面的问题，不是设计层面的问题。

AI Agent 不会拒绝工作，传统软件工程的所有经验和设想都可以在 *AI Agent* 开发中找到用武之地。核心问题从“如何让人类愿意执行流程”转变为“如何通过强制约束机制确保 *AI Agent* 完整执行流程”。

4.3 新原则

基于修正后的理解，我们提出了四项核心原则：

原则	说明	传统方式的区别
强制约束 > 建议约束	不是“应该做”，而是“必须做”	依赖人类自觉
系统自动验证 > 人工检查	系统自动发现缺口	依赖人类注意力
追溯关联 > 独立文档	需求-设计-代码-测试之间强制关联	文档独立存在
持久化记忆 > 依赖记忆	强制记录，自动加载	依赖人类记忆

第五章：动态 Checklist 机制设计

5.1 静态 checklist 的问题

传统评审 checklist 是静态的：同一份 checklist 适用于每个项目、每个文档、每个需求。这造成两个问题：

1. 噪音：许多 checklist 项目与被评审的具体文档无关
2. 缺口：文档特有的关键项目可能缺失

5.2 动态 Checklist 生成

我们提出了一个动态 checklist 机制，根据文档的实际内容生成评审项目：

5.3 动态 Checklist 示例

对于 FR-MEMORY-003 需求（缺失的 session_start 功能），动态 checklist 会生成：

第六章：人机协作模式

6.1 我们展示的模式

这次调查展示了一种新的人机协作模式：

1. **人类发起**：人类注意到 AI Agent 行为中的问题
2. **系统性调查**：人类和 AI Agent 一起追溯整个流程中的问题
3. **根因分析**：AI Agent 应用结构化思维（5-Why）寻找根本问题
4. **人类质疑与修正**：人类专家质疑 AI 的假设，引导走向正确方向
5. **共同设计**：人类和 AI Agent 共同设计改进机制
6. **文档化**：AI Agent 将整个过程捕获为可传递的知识

6.2 互补的优势

| 人类优势 | AI Agent 优势 |

|-----|-----|

| 基于经验的上下文理解 | 结构化分析思维 |

对微妙问题的识别	系统性调查
创造性问题解决	完整文档化
对什么"重要"的判断	明确指令的执行
质疑隐含假设的能力	执行明确约束的能力

6.3 人类的核心价值

这次探索最重要的发现是人类在 **AI** 时代不可替代的价值：

- 不是与 **AI** 竞争技术能力，而是提供 AI 所缺乏的判断力和质疑精神
- 不是关注技术细节，而是直击问题的核心
- 不是执行具体任务，而是确保任务方向的正确性

在这次案例中，你（人类专家）没有查看 `grep` 命令的结果，没有阅读代码实现，但你凭借常识判断出"人会主动发现问题"这个假设是错误的。这直接引导我们找到了真正的根因：不是"AI 不如人会主动"，而是"机制必须从期望主动转变为强制执行"。

第七章：结论与启示

7.1 关键发现

1. 传统软件工程的设计假设本身是合理的。多人协作确实需要流程控制、文档规范、质量门禁。问题在于人类员工普遍抵触文档工作，导致流程失去载体——这是人类执行层面的问题，不是设计层面的问题。

2. **AI Agent** 可以完美执行传统软件工程流程。AI 不会拒绝工作、不会消极抵抗，所有传统软件工程的经验和设想都可以在 AI Agent 开发中找到用武之地。
3. **AI Agent** 的问题是技术限制，不是主观意愿。context rotating（上下文丢失）和幻觉导致 AI Agent 可能偏离指令，但这与人类的主观抵触完全不同，可以通过技术手段解决。
4. **核心问题转变为**：如何通过强制约束机制确保 AI Agent 完整执行流程，而非如何让人类愿意执行流程。
5. **人类的核心价值依然不可替代**。虽然传统软件工程流程可以直接应用于 AI Agent，但人类能够质疑隐含假设、直击问题核心、引导 AI 走向正确方向。

7.2 对行业的启示

这次调查的影响超越 oc-collab：

- 对于 **AI 开发框架**：必须包含强制约束机制，而不仅仅是建议
- 对于 **AI 项目管理**：必须实施系统性验证，而不仅仅是文档存在性检查
- 对于 **AI 质量保证**：必须针对 AI Agent 行为模式设计，而非人类模式
- 对于 **AI 政策和治理**：必须认识到 AI Agent 的独特特征并设计适当的控制

7.3 前行之路

缺失的 session_start 功能不仅仅是一个 bug——它是 AI Agent 时代软件工程根本挑战的一扇窗口。通过系统性地调查这个缺口，我们已经开始绘制适合 AI Agent 的软件工程实践领域。

这份论文记录的不仅仅是一个解决方案，更是一种**思考问题的方式**——如何通过人机协作，发现并修正隐含的假设，从而找到真正有效的解决方案。

探索的旅程仍在继续。随着 AI Agent 变得越来越有能力、越来越普遍，对适当工程实践的需求只会增长。这份论文是对那场持续对话的贡献——记录一个小的缺口如何带来大的洞察，以及人机协作如何系统性地发现和限制我们集体潜力的隐藏假设。

附录：方法论

A.1 5-Why 技术

5-Why 技术是一种根因分析方法，通过反复问“为什么？”（通常五次）从直接症状深入到根本原因。

在我们的调查中：

1. 为什么功能缺失？→ 没有设计，没有测试，没有代码
2. 为什么没有设计/测试？→ 需求不清晰，没有被当作独立功能
3. 为什么评审没发现？→ 流程只检查文档存在
4. 为什么流程这样设计？→ 假设人类会验证功能
5. 为什么假设对 AI 失败？→ **这是错误的问题**

关键转折：在第五步时，人类专家质疑了前四步的隐含假设，指出“人会主动发现问题”本身就是错误的。这引导我们找到了真正的根因：机制必须从“期望主动”转变为“强制执行”。

A.2 动态 Checklist 算法

附录： oc-collab 项目背景

B.1 oc-collab 是什么

oc-collab (OpenCode Collaboration) 是一个双 **Agent** 协作开发框架，基于 OpenCode 平台实现。它的核心目标是解决 AI Agent 独立开发时的混乱问题，通过引入产品经理 (Agent 1) 和开发 (Agent 2) 的分离式协作，确保开发过程规范化、可追溯、可复用。

项目定位： 为 AI Agent 提供类似传统软件工程的流程约束，但不依赖人类监督。

核心价值：

- 双代理协作：产品 + 开发，互相监督
- 里程碑签署：质量门禁，不跳过
- 状态管理：单一事实来源
- 问题追踪：不重复踩坑
- 记忆机制：记得教训

B.2 v2.2.0 功能清单

oc-collab v2.2.0 于 2026-02-02 发布，包含以下核心功能：

| 模块 | 功能 | 说明 |

|-----|-----|-----|

| Agent 管理 | agent_manager.py | Agent 角色系统 (6 种类型) |

项目管理	project_manager.py	任务分配、依赖管理
资源锁	resource_lock.py	资源锁机制
会议管理	meeting_manager.py	会议管理
用户故事	story_manager.py	用户故事管理
签署引擎	signoff.py	签署流程
状态管理	state_manager.py	项目状态追踪
记忆机制	memory_manager.py	问题模式、决策历史、经验教训
测试覆盖：164 测试，88% 覆盖率。

B.3 协作流程详解

oc-collab 的协作流程基于传统软件工程的阶段划分：

各阶段文档要求：

阶段	文档位置	文档类型
需求	docs/01-requirements/	requirements_v.md, requirements_review_.md
设计	docs/02-design/	detailed_design_.md, design_review_.md
测试	docs/03-test/	blackbox_test_cases_.md, REVIEW_REPORT*.md
部署	docs/04-changelog/	change_log.md, RELEASE_NOTES_*.md

B.4 核心命令参考

命令	说明
oc-collab init <project_name>	初始化协作项目
oc-collab status	查看当前协作状态
oc-collab switch <1 2>	切换 Agent 角色
oc-collab advance -p <phase>	推进到指定阶段

oc-collab signoff <stage>	签署确认
oc-collab review <type>	评审功能
oc-collab history	查看协作历史

B.5 项目结构

B.6 签署机制详解

oc-collab 的签署机制是质量门禁的核心：

签署条件：

1. 双方必须都完成签署
2. 签署前必须通过自动检查
3. 签署后状态更新为 "APPROVED"

签署流程：

B.7 记忆机制详解

oc-collab 的记忆机制用于解决 Agent 会话隔离问题：

记忆类型	文件位置	内容
问题模式	state/memory/patterns.yaml	已知问题及其解决方案
决策历史	state/memory/decisions.yaml	重要决策及其理由
经验教训	state/memory/lessons.yaml	项目经验总结
待处理事项	state/memory/pending.yaml	当前待办

Compaction 保护：Compaction 操作前会导出所有记忆文件，确保不丢失关键信息。

B.8 相关文档链接

本文档中引用的 oc-collab 项目文档：

文档 路径
----- -----
需求文档 docs/01-requirements/requirements_v2.2.0.md
概要设计 docs/02-design/OUTLINE_DESIGN_v2.2.0.md
详细设计 docs/02-design/detailed_design_v2.1.0.md
测试用例 docs/03-test/blackbox_test_cases_v2.2.0.md
测试报告 docs/03-test/blackbox_test_results.md
MEMO-2026-02-003 docs/00-memos/MEMO-2026-02-003_oc-collab_Core_Design_Philosophy.md
MEMO-2026-02-004 docs/00-memos/MEMO-2026-02-004_AI_Agent_Engineering_Process.md

关于本文

标题：AI Agent 时代的人机协同软件工程：一次深度探索

作者：oc-collab 软件Agent 1（产品经理兼测试，执笔）+ 人类专家（刘臻）

日期：2026-02-03

版本：1.0

语言：中文

文档编号：TREATISE-2026-02-001

创建日期：2026-02-03

状态：终稿

本文记录了一次真实的人机协作探索过程，展示如何通过人类专家的质疑和引导，发现传统软件工程在 *AI Agent* 场景下的真正价值。核心发现是：传统软件工程的设计假设本身是正确的，人类的问题是主观抵触，*AI* 的问题是技术限制；人类的核心价值在于质疑隐含假设、直击问题核心、引导 *AI* 走向正确方向。